

Tool Use and Deterministic Control

Day 2 — give the Bookshelf assistant a tool, control when it fires, and decide the rest in code

Today's chapters

- 01** Why a model needs tools
- 02** Two kinds of tool
- 03** Defining a tool
- 04** Controlling when it fires
- 05** When selection goes wrong
- 06** Determinism, in code

01

Why a model needs tools

A model produces text. It cannot read today's stock, change a record, or show where an answer came from — unless something outside it can. Part A runs the assistant with nothing.

01 Why a model needs tools

02 Two kinds of tool

03 Defining a tool

04 Controlling when it fires

05 When selection goes wrong

06 Determinism, in code

It can only say things; a tool lets it do or look

Look

A fact from a system, now: stock, a record, a search. Text alone is the training data or the request — nothing later.

Do

A change the model cannot make by talking: reserve a copy, open a ticket, send a message. A sentence is not a row.

Show

A result your code produced — an id, a count, a citation — that the model reports instead of invents.

EXAMPLE · agent.py printout, offline run (scripted stand-in)

```
$ python agent.py --user daniel "Which form must ... chargeback?"
→ model turn 1 tools=[] mode=AUTO grounding=off
← model text: I don't have access to ... documents in this
  conversation, so I cannot say which form applies. ...
steps=1 calls=0 tokens=<n> cost=$<n>
```

THE RULE

A model turns text into text. A tool is a function your code owns that the model may ask for — and you run.

One vocabulary map, said once

In the topic list (Claude API)	On Gemini — google-genai	Lives in
tool_use block	functionCall part — response.function_calls	the model's turn
tool_result block	functionResponse part — Part.from_function_response(name, response)	your next user turn
tool_choice: auto · any · none · tool {name}	FunctionCallingConfig mode AUTO · ANY · NONE; allowed_function_names (with ANY)	tool_config
input_schema	parameters_json_schema — plain JSON Schema	FunctionDeclaration
strict: true — guaranteed conformance	no flag — the schema guides; VALIDATE in agent.py checks	your code
disable_parallel_tool_use	no switch — several functionCall parts may come back; PARALLEL in agent.py decides	your code
client tools · server tools	function_declarations you run · Tool(retrieval=...) Google runs	the request

THE RULE

Two Claude API features are your code on Gemini: strict conformance (VALIDATE) and one call per turn (PARALLEL).

The assistant with nothing

Goal

The kit as shipped: agent.env copied from agent.env.example with your project id, the two Day 1 ids and the bookshelf-api URL, and every switch at its Part A value — `TOOLS=off`, `GROUNDING=off`. Run the day's question. The `→ model` line says `tools=[]`: nothing was offered, so the model answers from nothing or says it cannot. Read the header lines once — caller, model, api — so you can tell a live run from anything else all day.

1

Set-up done: kit unzipped in Cloud Shell, agent.env filled in (first block), venv active, google-genai installed

2

Leave agent.env as shipped: `TOOLS=off`, `GROUNDING=off` — nothing to edit in this part

3

`python agent.py --user daniel "hello"` — read the three header lines: caller, model, api

4

`python agent.py --user daniel "Which form must a cardholder sign before a dispute can go to chargeback?"`

5

Read `→ model: tools=[]`. Read `← model`. Write down what it said and what it would have needed

THE RULE

Instructions: lab manual, Lab 1, Part A. Nothing to edit. Read the `→ model` line before the answer.

02

Two kinds of tool

Some tools you run; some Google runs inside the request. The Day 1 data store is the second kind — and it takes the same filter string, chosen in code.

- 01 Why a model needs tools
- 02 Two kinds of tool**
- 03 Defining a tool
- 04 Controlling when it fires
- 05 When selection goes wrong
- 06 Determinism, in code

Yours to run, or run for you inside the request

CLIENT-SIDE

```
Tool(function_declarations=[...])  
← model function_call reserve_book({... "copies": 2 ...})  
⚙ run reserve_book(...) → {"reservation_id": ..., ...}  
← function_response reserve_book {...}  
Your code ran it. Your code could have refused.
```

SERVER-SIDE

```
Tool(retrieval=Retrieval(vertex_ai_search=...))  
← model text: Every dispute ... requires ... form FRM-CD-7 ...  
   grounding: [1] Credit Card Dispute ... Policy (gs://...)  
   grounding: server-side retrieval — no call from our  
   side  
Google ran it inside the request. Nothing to refuse.
```

Which kind, when

Server-side: the data is in Google's service and nothing changes — search, grounding. You attach the tool; the service runs it; the reply arrives already informed, with grounding metadata. Client-side: the code is yours, the action changes a record, or you must be able to refuse it — your loop sees every call before it runs. A reservation is always client-side.

THE RULE

Server-side: nothing runs on your side — and nothing can be gated there. Anything that changes a record is client-side.

Day 1's store becomes a server-side tool with the caller's filter

AGENT.PY — GROUNDING=ON ADDS THIS TOOL

```
ds = (f"projects/{PROJECT_ID}/locations/global/"
      f"collections/default_collection/dataStores/"
      f"{DATASTORE_ID}")
tools.append(Tool(retrieval=Retrieval(
    vertex_ai_search=VertexAISearch(
        datastore=ds,
        filter=CALLERS[self.user]["filter"])))
```

The same string as yesterday

answer.sh --user daniel sent it in
searchSpec.searchParams.filter. agent.py --user
daniel sends it in VertexAISearch(filter=...).
callers.json holds it. The model never sees it.

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART B

```
--user daniel  grounding=on  tools=[]
  grounding: [1] Credit Card Dispute ... Policy (gs://...)
  grounding: server-side retrieval — no call from our side
--user priya   grounding=on  tools=[]
  grounding: no chunks returned (nothing passed the
  caller's filter)
```

What --user decides

The caller, the clearance, and so the filter — in
code, from the command line. Part F shows what
happens when the model tries to name a different
caller.

THE RULE

The filter is the same string as yesterday — chosen in code from the caller, sent with the tool, never written by the model.

The Day 1 store as a server-side tool

Goal

One switch: GROUNDING=on in agent.env. agent.py adds the Day 1 data store as a retrieval tool with the caller's filter string — read the line in callers.json before you run. Ask the day's question as daniel: a cited answer, grounding lines, and no  run line — the retrieval ran on Google's side. Ask it as priya: her filter stops at internal, so the confidential policy is out of reach and the answer says so. Then set GROUNDING=off again for Part C.

1

agent.env: GROUNDING=on (TOOLS stays off)

2

callers.json: read daniel's and priya's filter strings — the same two as yesterday

3

python agent.py --user daniel "Which form must a cardholder sign before a dispute can go to chargeback?" — the grounding: lines; no  run

4

python agent.py --user priya "..." — no chunks passed the filter; the refusal

5

agent.env: GROUNDING=off again before Part C

THE RULE

Instructions: lab manual, Lab 1, Part B. One switch on, two callers, then the switch off again.

03

Defining a tool

A tool is three things: a name the code looks up, a description the model reads, and a schema both must obey. Then the loop: `functionCall` in, `functionResponse` out, until the model answers in text.

- 01 Why a model needs tools
- 02 Two kinds of tool
- 03 Defining a tool**
- 04 Controlling when it fires
- 05 When selection goes wrong
- 06 Determinism, in code

Three parts, one JSON object, one FunctionDeclaration

TOOLS.JSON — ONE ENTRY, AS SHIPPED

```
{
  "name": "lookup_stock",
  "description": "Read-only: how many copies of one document in the Bookshelf library are available right now. Use for questions about stock or copies left.",
  "parameters": {
    "type": "object",
    "properties": {
      "book_id": {
        "type": "string",
        "description": "Document id, e.g. PRC-KYC-003"
      }
    },
    "required": ["book_id"]
  }
}
```

```
FunctionDeclaration(name=d["name"], description=d["description"],
  parameters_json_schema=d["parameters"])
```

Part	Who reads it
name	the model writes it in functionCall; your registry looks it up
description	the model, at the moment of choosing
parameters	the model, to shape the arguments; your code, to check them

tools.json is the whole surface the model sees. No URL, no credential, no retry policy reaches it — only these three fields, on every turn.

THE RULE A tool is three things: a name the code looks up, a description the model reads, a schema both must obey.

Write it for the moment of choice: what, when, what it changes

WEAK

"description": "does things"

Nothing about what, when, or what it changes. Any tool whose description mentions copies now looks like a better match for "reserve two copies".

STRONG

"description": "Reserve copies of a document in the Bookshelf library for a member of staff. Use when the caller asks to reserve, hold or book copies. This changes library records."

What · when (the caller's words) · what it changes.

EXAMPLE · agent.py printout, offline run (scripted stand-in) — RESERVE_BOOK DESCRIBED AS "DOES THINGS"

```
message: Reserve two copies of PRC-KYC-003 for daniel
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book]
← model function_call lookup_stock({"book_id": "PRC-KYC-003"})
⚙ run lookup_stock(...) → {..., "copies_available": 3, "copies_total": 3}
← model text: PRC-KYC-003 — ... : 3 of 3 copies available.
```

THE RULE

Cover the name and read the description alone: could someone pick this tool for the right request and refuse it for the wrong one?

Close every field you can: required, enum, integer with bounds

RESERVE_BOOK — PARAMETERS (TOOLS.JSON, PART C)

```
"parameters": {"type": "object",
  "properties": {
    "book_id": {"type": "string",
      "description": "Document id, e.g. PRC-KYC-003"},
    "copies": {"type": "integer",
      "minimum": 1, "maximum": 3,
      "description": "Copies to reserve, 1 to 3"},
    "caller": {"type": "string",
      "enum": ["priya", "daniel", "mlro"],
      "description": "Who the reservation is for"}},
  "required": ["book_id", "copies", "caller"]}
```

Constraint	Buys you
required	no missing field to guess about
enum	a closed set — nothing to parse
integer + min/max	a number, in the library's range
description on each	the model reads these too

On Gemini the schema is guidance to the model, not a guarantee: Part F's "five copies" comes back as copies: 5. The same schema, read by agent.py (VALIDATE=on), is the guarantee. Two readers, one file.

THE RULE The schema is the model's instructions and your code's checklist — close every field you can.

The model's turn, your result, the same name

THE TWO PARTS, IN THE SDK

```
resp = client.models.generate_content(  
    model=MODEL, contents=contents, config=config)  
parts = []  
for fc in resp.function_calls or []:  
    result = run(fc.name, dict(fc.args))    # your code  
    parts.append(Part.from_function_response(  
        name=fc.name, response=result))    # a dict  
contents.append(resp.candidates[0].content)  
contents.append(Content(role="user", parts=parts))  
# ... then generate_content again, same tools
```

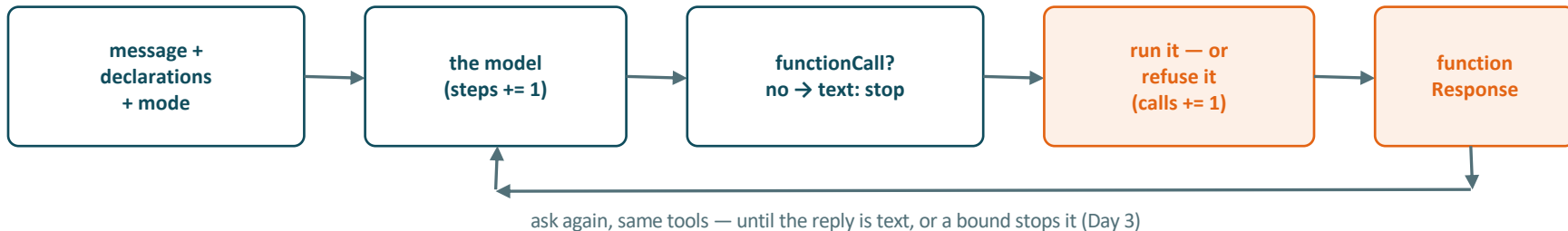
EXAMPLE · OFFLINE RUN

```
← model function_call  
  reserve_book({"book_id":  
    "PRC-KYC-003", "copies": 2,  
    "caller": "daniel"})  
⚙ run reserve_book(...) →  
  {"reservation_id": ...,  
    "copies_available": 1, ...}  
← function_response  
  reserve_book {...}  
← model text: Reserved 2 of  
  PRC-KYC-003 ... 1 left.
```

An error is a result. "needs approval", "invalid args", "HTTP 422" go back as a functionResponse with an error key, and the model reports them. The loop never throws at the model; it tells it.

THE RULE Echo the model's turn, answer every functionCall with a functionResponse of the same name, then ask again.

Ask, run what it asks for, tell it, ask again



EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART C

```

→ model turn 1 tools=[lookup_stock, ..., reserve_book]
← model function_call reserve_book({... "copies": 2 ...})
⚙️ run reserve_book(...) → {"reservation_id": ..., ...}
← function_response reserve_book {...}
→ model turn 2 tools=[lookup_stock, ..., reserve_book]
← model text: Reserved 2 of PRC-KYC-003 ... 1 left.
steps=2 calls=1 tokens=<n> cost=$<n> elapsed=<n>s
  
```

Read the last line first

- steps = model calls
- calls = tool runs that executed
- steps=1 calls=0: text only
- steps=2 calls=1: one tool round
- The loop decides: run, refuse, stop

THE RULE

The model proposes; the loop runs, refuses, or stops. It ends when the reply is text — or when your bound says so.

Define `reserve_book` and watch one round

Goal

Two switches and one edit. `agent.env`: `TOOLS=on` (the tools in `tools.json` are now sent), `GROUNDING=off`.
`tools.json`: add `reserve_book` after `search_bookshelf` — the name, the description from the manual, and the schema: `book_id` string, `copies` integer 1 to 3, `caller` enum of the three ids. Then one reservation and one stock question, and the four lines of a tool round: `functionCall`,  `run`, `functionResponse`, the confirmation. `lookup_stock` was already there; it shows one copy left.

1

`agent.env`: `TOOLS=on`, `GROUNDING=off`

2

`tools.json`: add the `reserve_book` entry after `search_bookshelf` (mind the comma) — Appendix A has the final file

3

`python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel"`

4

Read the four lines: `← model function_call` ·  `run` · `← function_response` · `← model text`

5

`python agent.py --user daniel "How many copies of PRC-KYC-003 are left?"` — `lookup_stock`: 1 of 3

THE RULE

Instructions: lab manual, Lab 1, Part C. One edit in `tools.json`, then read the four lines of the round.

04

Controlling when it fires

The model chooses when a tool fires — unless you take the choice away. Three modes, a forced call and its cost, and what to do when two calls come back at once.

- 01 Why a model needs tools
- 02 Two kinds of tool
- 03 Defining a tool
- 04 Controlling when it fires**
- 05 When selection goes wrong
- 06 Determinism, in code

Three modes and a list: FunctionCallingConfig

Topic list	Gemini	The model...
auto	mode=AUTO	may call a tool, or answer
none	mode=NONE	may not call — talk only
any	mode=ANY	must call one of the tools
specific tool	mode=ANY + allowed_function_names=[name]	must call that one

AGENT.PY — MODE AND ALLOWED FROM AGENT.ENV

```
tool_config = ToolConfig(
    function_calling_config=FunctionCallingConfig(
        mode=FunctionCallingConfigMode[mode], # AUTO|ANY|NONE
        allowed_function_names=allowed # only with ANY
        if mode == "ANY" and allowed else None))
```

EXAMPLE · OFFLINE RUN

```
$ MODE=NONE python agent.py \
  --user daniel "Reserve two ..."
→ model turn 1 tools=[]
  mode=NONE grounding=off
← model text: I can't take that
  action in this conversation. ...
  steps=1 calls=0

$ MODE=ANY ALLOWED=reserve_book \
  python agent.py --user daniel hello
→ model turn 1 tools=[]
  mode=ANY allowed=[reserve_book]
← model function_call
  reserve_book({...})
```

THE RULE

AUTO lets the model choose; **NONE** takes tools off the table; **ANY** forces one — with a list of one, a specific tool.

ANY on "hello": a call the request never needed

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART D

```
$ MODE=ANY python agent.py --user daniel hello
→ model turn 1 tools=[lookup_stock, search_bookshelf,
  reserve_book] mode=ANY grounding=off
← model function_call lookup_stock({"book_id": "POL-CR-014"})
⚙️ run lookup_stock(...) → {..., "copies_available": 3, ...}
← function_response lookup_stock {...}
→ model turn 2 tools=[...] mode=AUTO (ANY applied to
  the first turn only) grounding=off
← model text: Hello. I called lookup_stock because a
  tool call was required: {...}
steps=2 calls=1 tokens=<n> cost=$<n>
```

What the forced call cost

- A tool run nobody asked for — with ALLOWED=reserve_book, a reservation
- Two model turns instead of one, and the tool run: read the last line
- An argument the model had to invent

First turn only

agent.py applies ANY to turn 1 and AUTO after — the turn 2 line says so. Kept on every turn, the model could never answer in text and the loop would run to MAX_STEPS.

THE RULE

mode=ANY guarantees a functionCall comes back — and nothing about whether it was needed. Force on the first turn, then let go.

Two functionCalls in one turn; the loop decides what runs

PARALLEL=ON

```
← model function_call reserve_book({... "PRC-KYC-003" ...})
← model function_call reserve_book({... "PRC-CR-030" ...})
⚙ run reserve_book(...) → {"reservation_id": ...}
⚙ run reserve_book(...) → {"reservation_id": ...}
← function_response reserve_book {...}
← function_response reserve_book {...}
→ model turn 2 ...
steps=2 calls=2
```

PARALLEL=OFF

```
← model function_call reserve_book({... "PRC-KYC-003" ...})
← model function_call reserve_book({... "PRC-CR-030" ...})
⚙ run reserve_book(...) → {"reservation_id": ...}
⚙ run reserve_book(...) → blocked: one action per
  turn (PARALLEL=off)
← function_response reserve_book {"error": "one action..."}
→ model turn 2 ...
← model function_call reserve_book({... "PRC-CR-030" ...})
⚙ run reserve_book(...) → {"reservation_id": ...}
steps=3 calls=2
```

Disable when the calls change records and the second should see the first's result, when each needs its own approval, or when the tool is rate-limited. Leave on for independent read-only calls.

THE RULE

Gemini may return several calls at once and has no switch to stop it — one action per turn is a decision your loop makes.

Take the choice away, then give it back

Goal

Five commands, no file edits: each switch is typed in front of python and applies to that run only. MODE=NONE on a reservation request: talk only, no  run. MODE=ANY on "hello": a call the request never needed — read what it cost on the last line. MODE=ANY ALLOWED=reserve_book on "hello": a forced reservation. Two documents in one request: two functionCalls in one turn. PARALLEL=off on the same request: the second is refused and comes back in the next turn.

1

MODE=NONE python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel" — mode=NONE, no  run

2

MODE=ANY python agent.py --user daniel "hello" — which tool did it force, and what did the last line say?

3

MODE=ANY ALLOWED=reserve_book python agent.py --user daniel "hello" — a reservation for a greeting

4

python agent.py --user daniel "Reserve one copy of PRC-KYC-003 and PRC-CR-030 for daniel" — two calls, one turn

5

PARALLEL=off ... the same message — blocked: one action per turn; steps=3 calls=2

THE RULE

Instructions: lab manual, Lab 1, Part D. Switches in front of the command, one run each. Bring the five last lines to the debrief.

Which line proved what

What you should have seen

Five runs, five last lines. NONE: steps=1 calls=0 and no  run — the tools were sent and could not be called. ANY on hello: steps=2 calls=1 — a turn and a tool run the request never needed, and with ALLOWED, a reservation. Two documents: two functionCall lines before any  run, steps=2 calls=2. PARALLEL=off: blocked: one action per turn, then the same call again on turn 2, steps=3 calls=2 — one extra turn bought one side effect at a time.

1

MODE=NONE: which line proves no tool could be called — and would a sentence in the prompt have proved it?

2

MODE=ANY on "hello": name the three costs on your printout. Which one changed a record?

3

Turn 2 said "ANY applied to the first turn only". What would happen to the loop without that rule?

4

PARALLEL=off cost one more turn. What did the model see before it re-issued the second call, and why does that matter for a reservation?

THE RULE

Mode is a request setting the API enforces; a prompt is a request to the model. Parallel is neither — it is your loop.

05

When selection goes wrong

The model picks from what it is given. Give it a description that says nothing, or eight tools that say the same thing, and the pick is wrong before the model has done anything wrong.

- 01 Why a model needs tools
- 02 Two kinds of tool
- 03 Defining a tool
- 04 Controlling when it fires
- 05 When selection goes wrong**
- 06 Determinism, in code

Eight tools, one request — the list made the pick wrong

EXAMPLE · agent.py printout, offline run (scripted stand-in) — PART E, SIX WAYS TO RESERVE

```
→ model turn 1 tools=[lookup_stock, search_bookshelf, reserve_book,
    reserve_books, book_reserve, reserve_copy, make_reservation,
    reserve_book_for_staff] mode=AUTO grounding=off
← model function_call reserve_book_for_staff({... "copies": 2 ...})
⚙ run reserve_book_for_staff(...) → blocked: no implementation for
  'reserve_book_for_staff' (declared in tools.json, missing in
  tools_impl.py)
← model text: I could not reserve PRC-KYC-003: no implementation ...
steps=2 calls=0 tokens=<n> cost=$<n>
```

Two costs, both on the printout

Every declaration is in the prompt on every turn — eight instead of three, twice. And a pick among look-alikes that no description can make right.

The rule before any tuning

One tool per action. Send only what the request might need (Day 3 scopes the list per task). Prune, then read the → model line: is every name there for a reason?

THE RULE

Every tool you send is read on every turn — and among look-alikes the wrong pick is the list's fault, not the model's.

NAMING COLLISIONS

One verb, one object, one tool — names the code can find

COLLISION

<code>reserve_book</code>	"Reserve copies of a document..."
<code>reserve_books</code>	"Reserve books for staff."
<code>book_reserve</code>	"Reserve a book from the..."
<code>reserve_copy</code>	"Reserve one or more copies..."
<code>make_reservation</code>	"Make a reservation for a..."
<code>reserve_book_for_staff</code>	"Reserve copies of a document..."

Six names, one action. The model picks; code knows one.

FIXED

<code>lookup_stock</code>	"Read-only: how many copies..."
<code>search_bookshelf</code>	"Search the approved policies..."
<code>reserve_book</code>	"Reserve copies of a document... This changes library records."

Three names, three actions. `verb_object`, one tool per action, no synonyms. Every name has an implementation.

Three habits, one check

`verb_object`. One tool per action. If two names could describe the same action, delete one. And the kit's check: a name in `tools.json` with no implementation is blocked before it runs — `"impl": "reserve_book"` on the entry maps a renamed tool back to its function.

THE RULE

The name is looked up twice — by the model in the request, by your code in the registry. Make the two lookups impossible to confuse.

Break the list three ways, then fix it

Goal

Three edits to tools.json, the same reservation request each time. Change reserve_book's description to "does things": the model picks the wrong tool. Put the description back and add five look-alikes with the same schema and a one-line description each: the model picks one that has no implementation. Delete the five: the reservation runs. The fixtures folder has the two broken files and the fixed one if you would rather copy than type.

1

tools.json: reserve_book's description → "does things" (or cp fixtures/tools.lab1e-bad.json tools.json)

2

python agent.py --user daniel "Reserve two copies of PRC-KYC-003 for daniel" — which tool ran?

3

Description back; add reserve_books, book_reserve, reserve_copy, make_reservation, reserve_book_for_staff (or cp ...lab1e-dupes.json)

4

Same command — read the → model list and the  run line: blocked: no implementation

5

Delete the five (or cp fixtures/tools.lab1e-fixed.json tools.json); same command — reserve_book runs

THE RULE

Instructions: lab manual, Lab 1, Part E. Same request three times; only tools.json changes. --reset between runs.

Three failures, and none of them the model's

What you should have seen

The same request three times. "does things": a wrong pick — the description gave the model nothing to choose with, and the name alone was not enough (or it was — say what yours did). Eight tools: a pick among six ways to reserve, one of which the code could run; the model's choice was reasonable and the list made it wrong. Three tools: the reservation ran. The model behaved the same way all three times: it read what it was given and chose.

1

Run 1: which of the three parts of the definition failed — name, description or schema? What would a reviewer have caught?

2

Run 2: the model chose `reserve_book_for_staff` (or another look-alike). Was that a wrong choice? Whose fault was the blocked line?

3

The `→` model line listed eight names on both turns. What did the five extra ones cost even though none was meant to run?

4

Write three rules for a `tools.json` review — one for descriptions, one for names, one for the length of the list.

THE RULE

The model reads what it is given and chooses. A wrong pick from a bad list is a list problem — fix the list, not the prompt.

06

Determinism, in code

The model's choice is sampled text and cannot be made deterministic. The system can: decide in code what happens to what the model proposes, and the same request gives the same outcome every time.

- 01 Why a model needs tools
- 02 Two kinds of tool
- 03 Defining a tool
- 04 Controlling when it fires
- 05 When selection goes wrong

06 Determinism, in code

Fix what happens to the model's choice, not the choice

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART F, THREE TIMES

```
$ APPROVE=off python agent.py --user daniel "Reserve one ..."  
← model function_call reserve_book({... "copies": 1 ...})  
⚙ run reserve_book(...) → blocked: needs approval (APPROVE=off)  
← function_response reserve_book {"error": "needs approval..."}  
← model text: I could not reserve PRC-KYC-003: needs  
  approval (APPROVE=off).  
steps=2 calls=0  
  
run 2 of 3: identical, except elapsed=  
run 3 of 3: identical, except elapsed=
```

The model's, and allowed to vary

- Which tool it proposes
- How it words the reply
- How it reports an error

Yours, and the same every time

- Whether a call runs (APPROVE)
- Whether the args are valid (VALIDATE)
- Who the caller is; when to stop

"The model decided" is not a sentence an auditor accepts. "The code refused because APPROVE=off" is — and it is only available because the decision was in code.

THE RULE

You cannot make the model's choice deterministic. You can make what happens to it deterministic — in code, the same every time.

A decision with a right answer gets a line of code

Decision	Where	In the kit
Which tool fits the request	the model	AUTO
Are the arguments valid	schema + code	VALIDATE
May this action run	code (a person, Day 3)	APPROVE
Who is the caller → filter	code	--user, callers.json
Needs no model at all	code (Day 3)	ROUTER
When to stop	code (Day 3)	MAX_STEPS ...
How to word the answer	the model	—

AGENT.PY — EXECUTE(): THE THREE LINES

```
if on("VALIDATE"):
    problem = validate_args(tool["parameters"], args)
if "caller" in args and args["caller"] != self.user:
    return {"error": f"caller must be {self.user}"}
if is_side_effect(tool):
    verdict = self.approval(args)
```

EXAMPLE · OFFLINE RUN — PART F

```
$ python agent.py --user priya \
    "Reserve one copy of PRC-KYC-003 for daniel"
← model function_call reserve_book(
    {... "caller": "daniel"})
⚙ run reserve_book(...) → blocked: caller
mismatch: model said 'daniel', this
session is 'priya'
```

THE RULE

If a decision has a right answer, it has a line of code. The prompt is for the decisions that do not.

A flag on Claude; your validator on Gemini

VALIDATE=OFF

```
$ python agent.py --user daniel "Reserve five copies ..."  
← model function_call reserve_book({... "copies": 5 ...})  
⚙ run reserve_book(...) → {"error": "HTTP 422 from  
bookshelf-api: copies: Input should be between  
1 and 3"}  
steps=2 calls=1  
A network call; the API did the checking.
```

VALIDATE=ON

```
$ VALIDATE=on python agent.py ... "Reserve five copies ..."  
← model function_call reserve_book({... "copies": 5 ...})  
⚙ run reserve_book(...) → blocked: invalid args:  
copies=5 > maximum 3  
← function_response reserve_book {"error": "invalid..."}  
steps=2 calls=0  
Nothing left the process; the message is yours.
```

What validate_args checks — thirty lines in agent.py

required present · no unexpected keys · type (2.0 accepted as 2) · enum · minimum and maximum. It does not make the model write valid arguments; it makes invalid ones not run.

THE RULE

A schema is advice to the model. Conformance is a guarantee — on Gemini, one your code makes, and one you can read.

Decide it in code, and get the same answer three times

Goal

Five commands, no edits. `VALIDATE=on` with "five copies": the schema says 3, the model writes 5, the loop blocks it before any call — `calls=0`; run it once without `VALIDATE` first and watch the API refuse instead. `APPROVE=off`: the reservation is blocked however it is asked — run it three times and compare the printouts line by line. Then `--user priya` asking to reserve for daniel: the model writes caller daniel, the loop blocks the mismatch. The caller, and so the filter string, was the session's all day.

1

`python agent.py --user daniel "Reserve five copies of PRC-KYC-003 for daniel"` — HTTP 422 from the API, `calls=1`

2

`VALIDATE=on` ... the same message — blocked: invalid args: `copies=5 > maximum 3`, `calls=0`

3

`APPROVE=off python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"` — blocked: needs approval

4

The same `APPROVE=off` command twice more — compare the three printouts, line by line

5

`python agent.py --user priya "Reserve one copy of PRC-KYC-003 for daniel"` — blocked: caller mismatch

THE RULE

Instructions: lab manual, Lab 1, Part F. Switches in front, no edits. Keep the three identical printouts for the debrief.

The outcome was never the model's to vary

What you should have seen

Five runs, three decisions made in code. "Five copies": with `VALIDATE=off` a network call and the API's refusal; with `VALIDATE=on` no call and your refusal — same schema, read by `agent.py` instead of only by the model. `APPROVE=off` three times: the blocked: line identical on all three, the model's wording free to differ. `--user priya` reserving for daniel: the model wrote what the message said, and the session decided — as the filter string was decided in Part B, and as `answer.sh --user` decided it yesterday.

1

Both "five copies" runs refused the reservation. Name what `VALIDATE=on` removed: a call, a dependency, a message — whose?

2

Three `APPROVE=off` printouts: which lines were identical and which were allowed to differ? What made the identical ones identical?

3

`priya` asked for daniel and the model wrote caller: "daniel". Why is that not the model's mistake — and where was the caller decided?

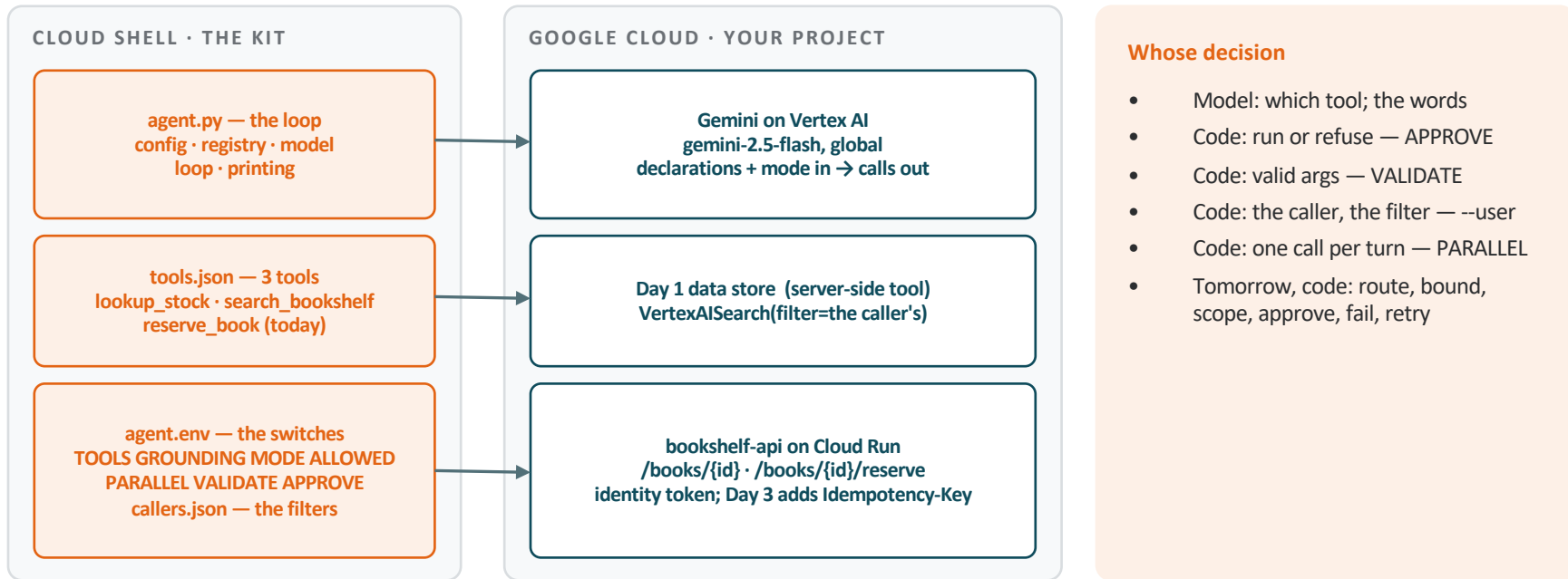
4

Part B's filter string, today's caller check, yesterday's `answer.sh --user`: one design decision. Say it in one sentence.

THE RULE

The model proposes; your code disposes — and the filter string was never the model's to choose.

One loop, two files, and every decision in its place



THE RULE

The model proposes; your code disposes. Tomorrow the same two files bound the loop, put a person in it, and survive a failure.